

The background features several overlapping circles in various shades of purple. A thin vertical purple line is positioned on the right side, with a small purple circle at its top end.

Reinforcement Learning Agent



01

Fundamentals RL Agent

What is Agent?

Agent

Agent agent, or simply referred to as Agent, is an entity that can autonomously perceive, think, make decisions, and act. It has the ability to navigate, learn, adapt, and grow in complex environments to achieve its established goals.

Type of Agent

- **Reinforcement learning agent** learns through behavioral consequences to maximize cumulative rewards;
- **Large language model agents** are adept at processing human language and performing text tasks such as response generation, summarization, and translation.

Markov Chain Fundamentals



Random Model

Markov Chain is a stochastic model that describes a sequence of events/states, where the probability of the next state depends only on the current state and has Markov properties, meaning that the future state is independent of the past state given the current state.



Memory free property

The memory free property of Markov Chain is manifested by the fact that given the current state, the probability of future states depends only on the current state and is not affected by past states, thereby simplifying the complexity of modeling complex systems.

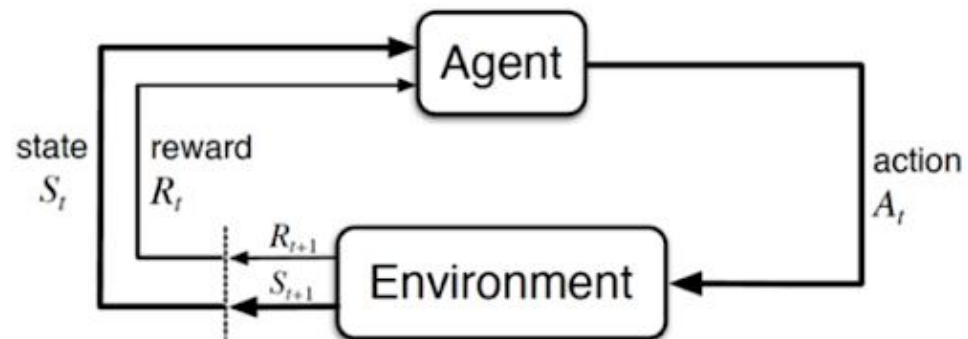
Markov Decision Process

Reinforcement learning framework

MDP reinforcement learning framework, including states, actions, transition probabilities, and rewards. Agents need to learn the optimal strategy to maximize cumulative rewards in a specific environment, and achieve optimal decisions through interaction with the environment.

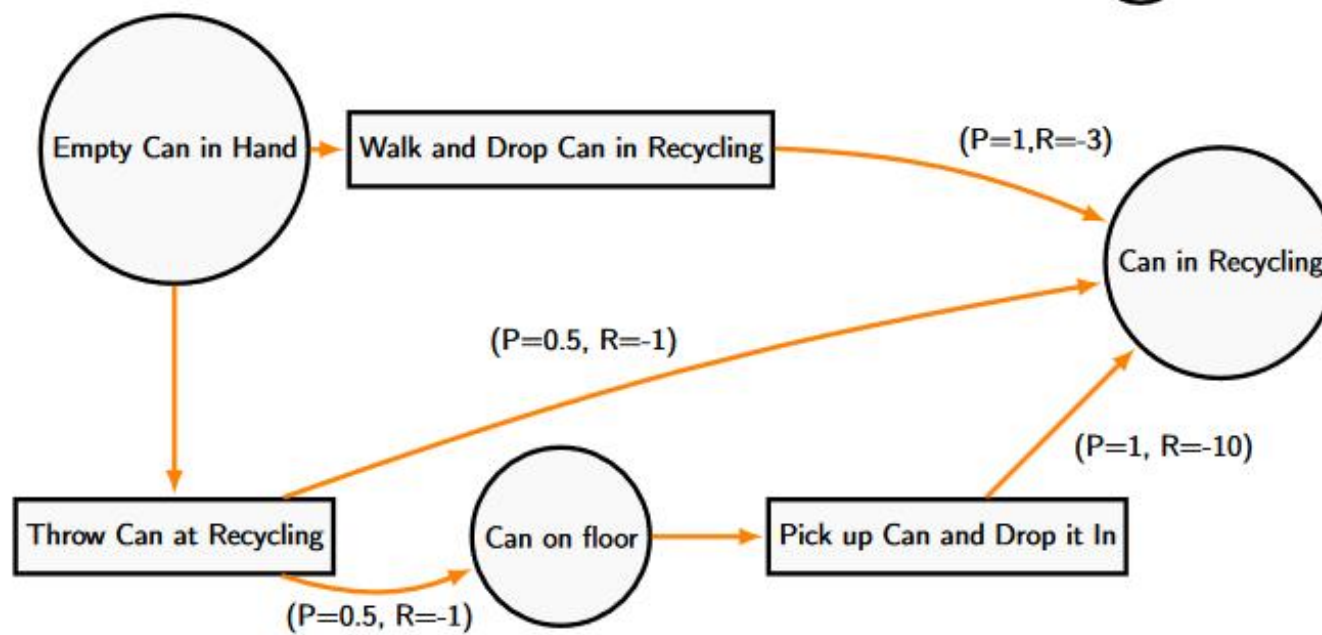
Strategic decision-making

Strategy is the mapping of states to actions, defining the behavior choices of agents in each state. The goal of reinforcement learning is to find a strategy that maximizes the cumulative return from the initial state to the termination state.



State

Action



Basic Principles of Reinforcement Learning

Feedback and Exploration

Reinforcement learning emphasizes a balance between exploration and utilization, discovering its effects through trying new operations and utilizing known operations with high returns. Meanwhile, feedback (rewards) may be delayed, so agents need to consider the long-term impact of their behavior.

Unsupervised learning

Without the need to label data, reinforcement learning learns the optimal strategy directly through interaction with the environment, without explicitly correcting suboptimal operations. This trial and error learning approach enables reinforcement learning to self improve and adapt in complex environments.

Overview of Deep Reinforcement Learning

01

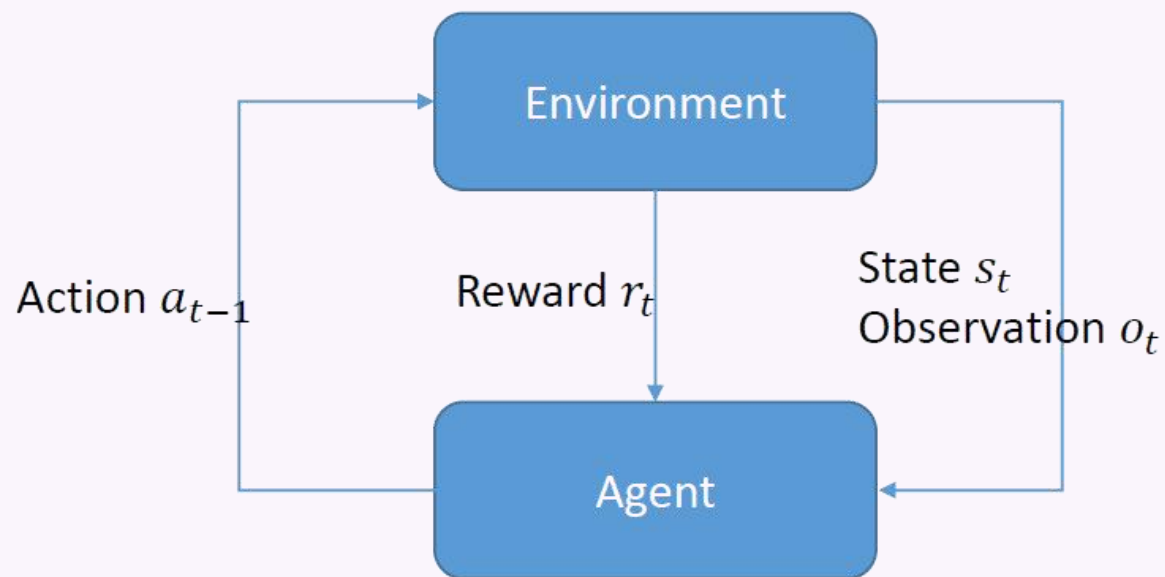
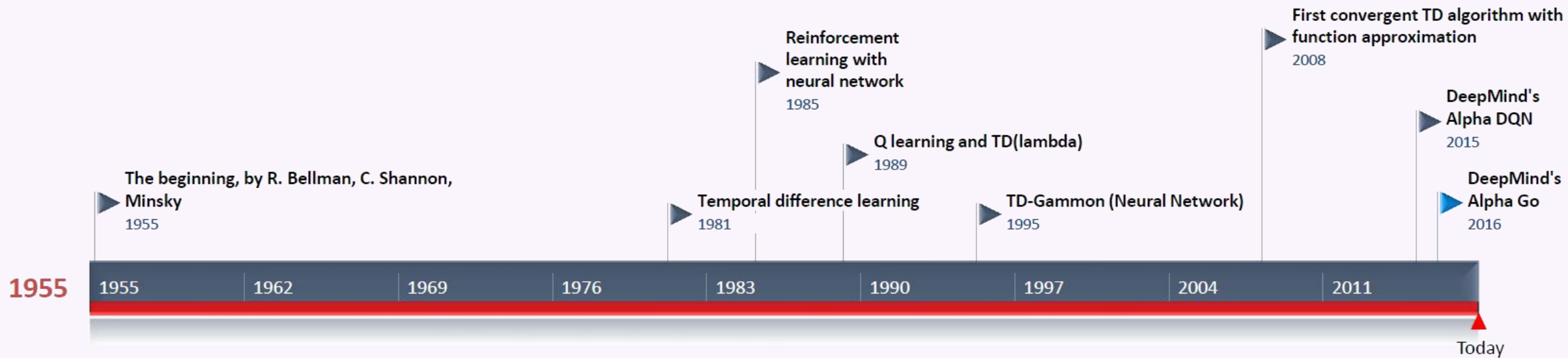
Deep integration

Deep reinforcement learning combined with deep learning can process unstructured data, such as video game screen pixels, and automatically learn complex features without manually designing a state space, providing powerful decision support for intelligent agents.

02

Widely used

Deep reinforcement learning has been applied in fields such as robotics, video games, natural language processing, and computer vision, driving the continuous development of artificial intelligence technology by processing unstructured data and achieving efficient decision-making.



Reinforcement Learning Model Architecture

01

State set

The state set defines all possible states of the environment and is one of the fundamental elements in reinforcement learning models. It covers a variety of state spaces from simple to complex, from finite to infinite.

02

Operation set

The operation set contains all possible operations that the agent can perform in each state. These operations can be discrete (such as pressing keys, moving positions) or continuous (such as adjusting parameters, changing speed).

03

Reward mechanism

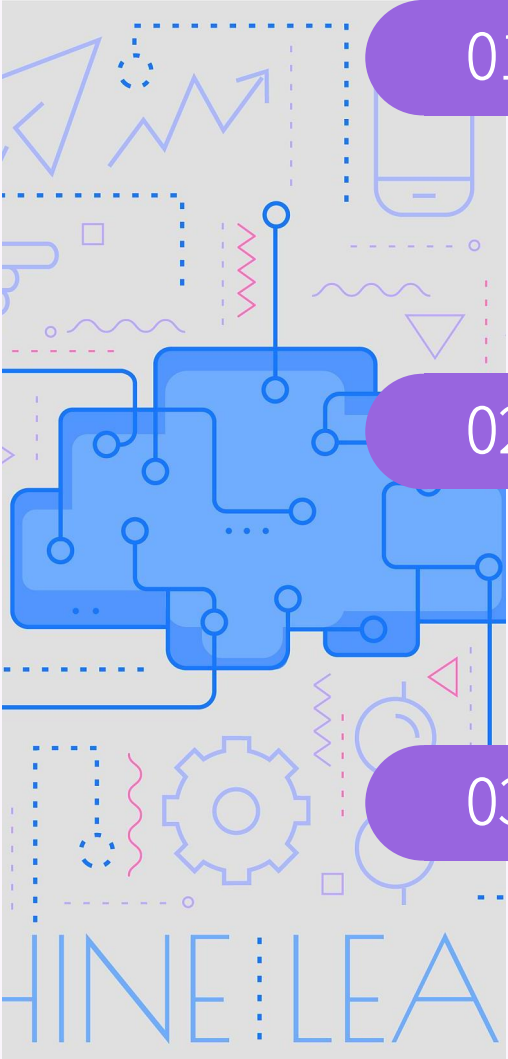
The reward mechanism describes the immediate reward obtained after transitioning from one state to another due to a specific operation. It reflects the immediate feedback and long-term trends of the environment on agent behavior.

04

Strategy Function

Strategy is the core function in reinforcement learning, which defines the behavior of agents in a given state. Strategy is typically represented as a mapping from state to operation, guiding agents to take corresponding actions.

The Development of Reinforcement Learning History



01

The origin of reinforcement learning

Reinforcement learning originated from trial and error learning in psychology, and later integrated optimal control theory in computer science, aiming to optimize decision-making processes through trial and error and feedback to solve complex problems.

02

Trial and error learning fusion

In the 1980s, trial and error learning was integrated with value function optimal control and time difference learning methods, laying a solid foundation for reinforcement learning and driving the field towards a more systematic and efficient direction.

03

Modern progress

Since the 1990s, with the introduction of algorithms such as Deep Q-Network, reinforcement learning has made significant progress in handling high-dimensional spaces and complex tasks, providing powerful tools for the field of intelligent decision-making.

RL application scenario analysis

Nonlinear control

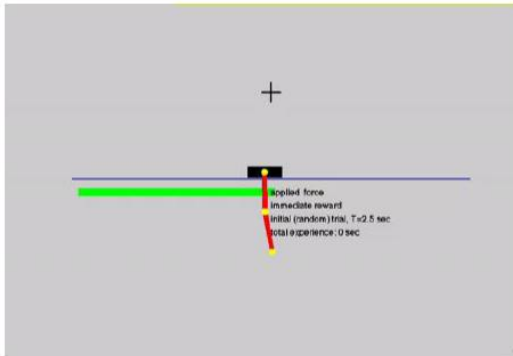
Nonlinear control is one of the key application areas of reinforcement learning, which learns and optimizes control strategies for complex systems through intelligent agents interacting with the environment, achieving high-precision and robust control effects.



Robot control

Robot control is another important application of reinforcement learning. By training intelligent agents, robots can learn tasks such as autonomous navigation and object manipulation, improve their intelligence level, and demonstrate broad application prospects.

RL Applications



Non-linear control



AlphaGo



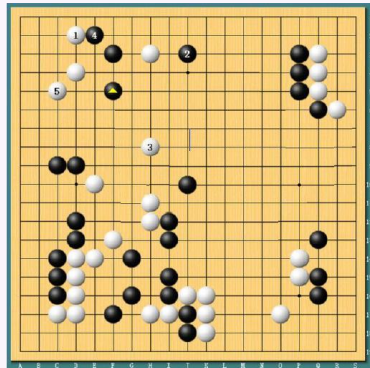
Robot



Play a variety of video games in Atari

RL Applications

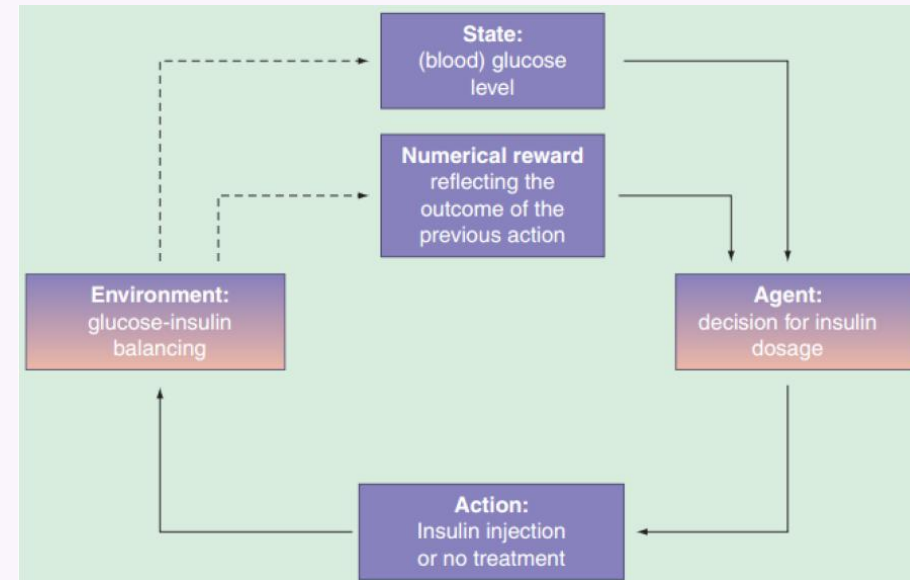
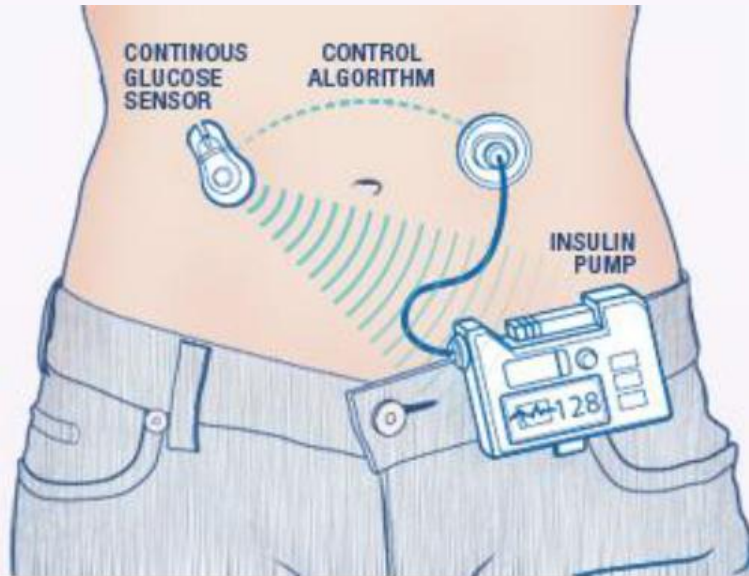
Application	Action	Observation	State	Reward
Playing Go (boardgame)	Where to place a stone	Configuration of board	Configuration of board	Win game: +1 Else: -1
Playing Atari (video games)	Joystick and button inputs	Screen at time t	Screen at times $t, t-1, t-2, t-3$	Game score increment
Conversational system	What to say to the user	What user says	History of the conversation	Task success: +10 Task fail: -20 Else: -1



Pancreatic insulin monitoring case

- **Application background:** Insulin monitoring is an important part of diabetes management. Through real-time monitoring of patients' blood glucose level, combined with reinforcement learning algorithm, you can intelligently adjust insulin dose and improve blood glucose control effect.
- **Algorithm classification:** Model based reinforcement learning algorithms (such as Dyna-Q) use environmental models for planning to reduce exploration costs; Model free algorithms, such as Q-Learning, learn by directly interacting with the environment.
- **Adaptive advantage:** Model based reinforcement learning algorithms can utilize models for systematic exploration and planning, improving sample efficiency; Model free algorithms adapt to environmental changes through random exploration strategies and have strong flexibility.
- **Practical application challenges:** When applying reinforcement learning to medical fields such as insulin monitoring, it is necessary to consider issues such as algorithm scalability, computational complexity, medical ethics, and safety to ensure the feasibility of the algorithm in practical applications.

Pancreatic Insulin Monitoring Example



Bothe M K, Dickens L, Reichel K, et al. The use of reinforcement learning algorithms to meet the challenges of an artificial pancreas[J].



02

Classification of reinforcement learning algorithms

Model based and Model free RL

Model Free RL

does not rely on environmental models, learns directly from experience, adapts flexibly to unknown environments, optimizes strategies through trial and error, and is suitable for complex dynamic environments, but has low sample efficiency and high computational costs.

Model Based RL

Utilizing environmental models to predict state transitions and rewards, efficiently plan actions, reduce exploration requirements, and is suitable for situations with known environmental dynamics. However, its generalization ability is limited and requires an accurate model.

Value and Strategy Fundamentals Method

01

Value Based RL

The numerical based RL method aims to optimize the value function $V(s)$, reflecting the expected maximum future reward in each state. Guiding strategies by solving $V(s)$ or $Q(s, a)$ is applicable to explicit state action value problems.

02

Policy Based RL

Policy based RL directly optimizes the policy function $\pi(s)$, defines proxy behavior, and does not rely on value functions. The random strategy does not converge to a definite value, but adapts flexibly to changes, and the Gaussian distribution is preferred for continuous operation space.

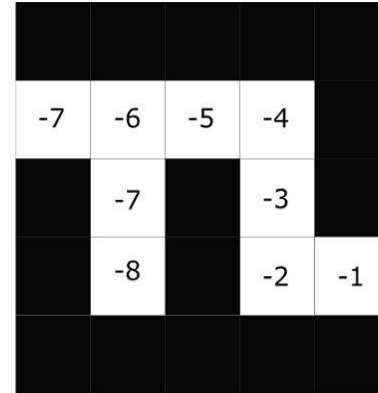
$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Expected

Reward
discounted

Given that state

Start



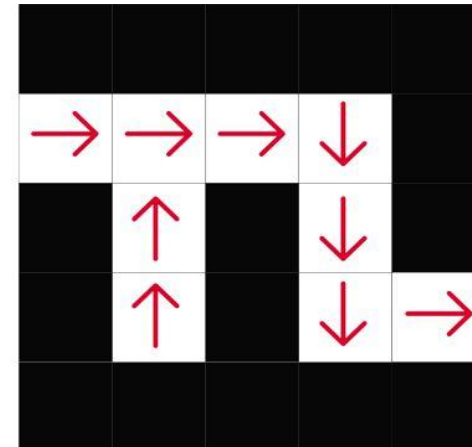
Goal

Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

Proba of

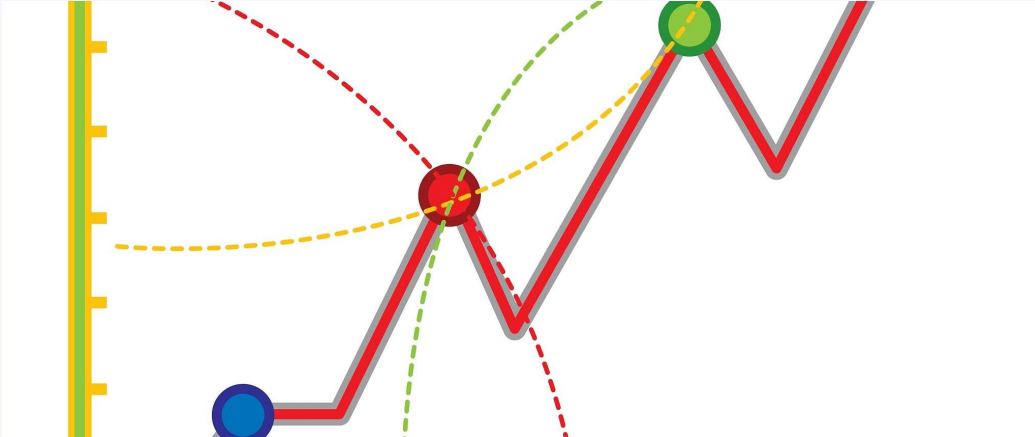
Take a
particular action
conditionned to
a state

Start



Goal

Value Based vs Policy Based

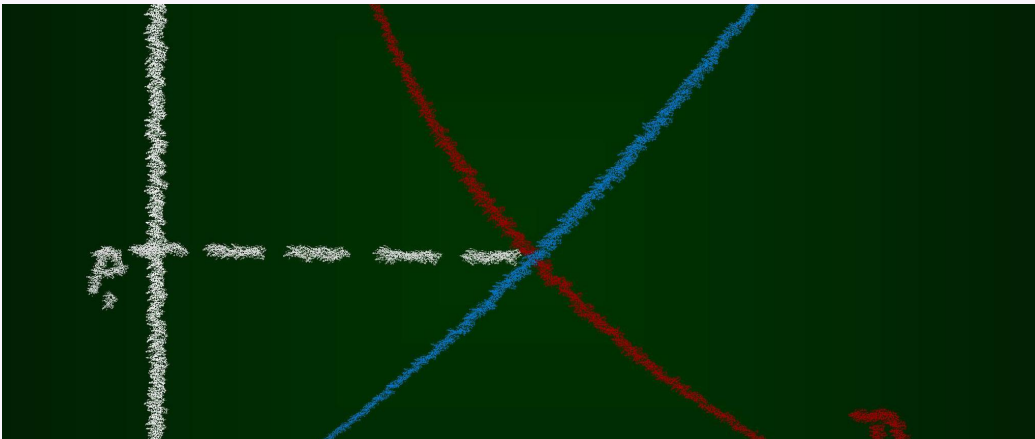


Value Based

In numerical RL, the goal is to optimize the value function $V(s)$, which is a function that indicates the maximum expected future reward that the agent will receive in each state, with input s and output $Q(s, a)$.

Policy Based

In policy based RL, we aim to directly optimize the policy function without using the value function $\pi(s)$, which defines the behavior of the agent at a given time, with input s and output $p(s, a)$.



Comparison of Same Strategy and Different Strategy

On Policy RL

- Only interacts with the same environment for learning, with samples generated by the same policy to avoid bias.
- Policy evaluation and improvement are closely integrated, making it suitable for intelligent agent control scenarios.
- However, sample efficiency is low and it is difficult to generalize to multi policy environments.

Off Policy RL

It can learn from its own and other environments, update with different policy data, has high sample efficiency, strong generalization ability, and can adapt to changing environments such as DQN and DDPG, but may introduce bias and require technical mitigation.

Analysis of SARSA Algorithm

01

Overview of SARSA

SARSA is a strategy reinforcement learning method that simultaneously learns value functions and strategies, and updates them using actual executed operations (on policies). Applicable to tasks that agents need to learn from the current strategy.

02

SARSA update rule

Update the value function based on the received reward and the value of the next state operation pair. Use bootstrapping method to estimate the value function to ensure that the strategy is consistent with the actual operation and improve learning efficiency.

03

Advantages and disadvantages of SARSA

SARSA is easy to implement and performs well in environments with fixed proxy strategies; However, its convergence speed may be slower and its performance may not be as good as some other reinforcement learning methods, such as Q-learning.

SARSA Algorithm

1. **Initialize** Q-values $Q(s, a)$ arbitrarily (e.g., to 0) for all states s and actions a .
2. **Repeat** for each episode:
 1. Initialize the starting state s .
 2. Choose an action a using the current policy (e.g., ϵ -greedy).
 3. **Repeat** for each step of the episode:
 1. Take action a , observe the reward R and the next state s' .
 2. Choose the next action a' using the current policy.
 3. Update the Q-value:
$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$$
 4. Set $s \leftarrow s'$ and $a \leftarrow a'$.
4. Until s is a terminal state.

SARSA Example

1. **Initialize Q-values:** $Q(s, a) = 0$ for all states and actions.

2. **Episode 1:**

- **Start State:** $s = (0, 0)$, choose action $a = \text{right}$.
- **Take Action:** Move to state $s' = (0, 1)$, receive reward $R = 0$.
- **Choose Next Action:** $a' = \text{right}$.
- **Update Q-value:**
$$Q((0, 0), \text{right}) \leftarrow Q((0, 0), \text{right}) + \alpha [R + \gamma Q((0, 1), \text{right}) - Q((0, 0), \text{right})]$$
$$Q((0, 0), \text{right}) \leftarrow 0 + 0.1 [0 + 0.9 \cdot 0 - 0] = 0$$
- **Next State:** $s = (0, 1)$, action $a = \text{right}$.

3. **Continue** the episode similarly, updating the Q-values using the SARSA update rule.

Q Learning Fundamentals



Q-learning algorithm

Q-learning is a model free reinforcement learning algorithm that optimizes strategies by iteratively updating the Q-table (state action value table). The core lies in using the Bellman equation and max operator to iteratively improve the accuracy of Q values.



Q-learning process

The algorithm process includes steps such as initializing the Q-table, selecting and executing operations, evaluating, and updating Q-values. By continuously selecting the best operation in the current state and updating the Q value, strategy optimization and improvement can be achieved.



Q-learning and DQN

Q-learning, as a traditional RL method, approximates discrete/continuous states through tables or functions; DQN introduces deep neural networks on this basis to achieve efficient processing of complex tasks such as high-dimensional visual inputs.

Q Learning Algorithm

1. **Initialize** Q-values $Q(s, a)$ arbitrarily (e.g., to 0) for all states s and actions a .
2. **Repeat** for each episode:
 1. Initialize the starting state s .
 2. **Repeat** for each step of the episode:
 1. Choose an action a using an exploration strategy (e.g., ϵ -greedy).
 2. Take action a , observe the reward R and the next state s' .
 3. Update the Q-value:
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$


Off-Policy updated
 4. Set $s \leftarrow s'$.
3. Until s is a terminal state.

Q Learning Example

1. **Initialize Q-values:** $Q(s, a) = 0$ for all states and actions.

2. **Episode 1:**

- **Start State:** $s = (0, 0)$, choose action $a = \text{right}$.
- **Take Action:** Move to state $s' = (0, 1)$, receive reward $R = 0$.

- **Update Q-value:**

$$Q((0, 0), \text{right}) \leftarrow Q((0, 0), \text{right}) + \alpha \left[R + \gamma \max_{a'} Q((0, 1), a') - Q((0, 0), \text{right}) \right]$$

$$Q((0, 0), \text{right}) \leftarrow 0 + 0.1 [0 + 0.9 \cdot 0 - 0] = 0$$

- **Next State:** $s = (0, 1)$.

3. **Continue** the episode similarly, updating the Q-values using the Q-learning update rule.

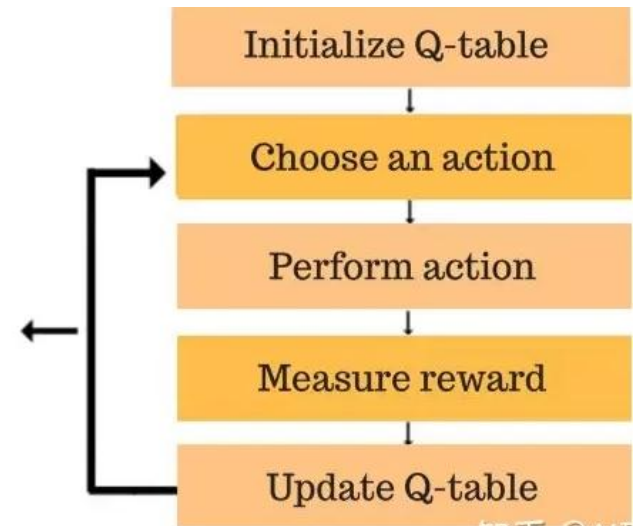
$$\text{New } Q(s,a) = Q(s,a) + \alpha [R(s,a) + \gamma \max_{a'} Q'(s',a') - Q(s,a)]$$

- New Q Value for that state and the action
- Learning Rate
- Reward for taking that action at that state
- Current Q Values
- Maximum expected future reward given the new state (s') and all possible actions at that new state.
- Discount Rate

Actions : ↑ → ↓ ←

Start				
Nothing / Blank				
Power				
Mines				
END				

After a lot of Iterations,
a good Q-table is ready



DQN

● Introduction to DQN Model

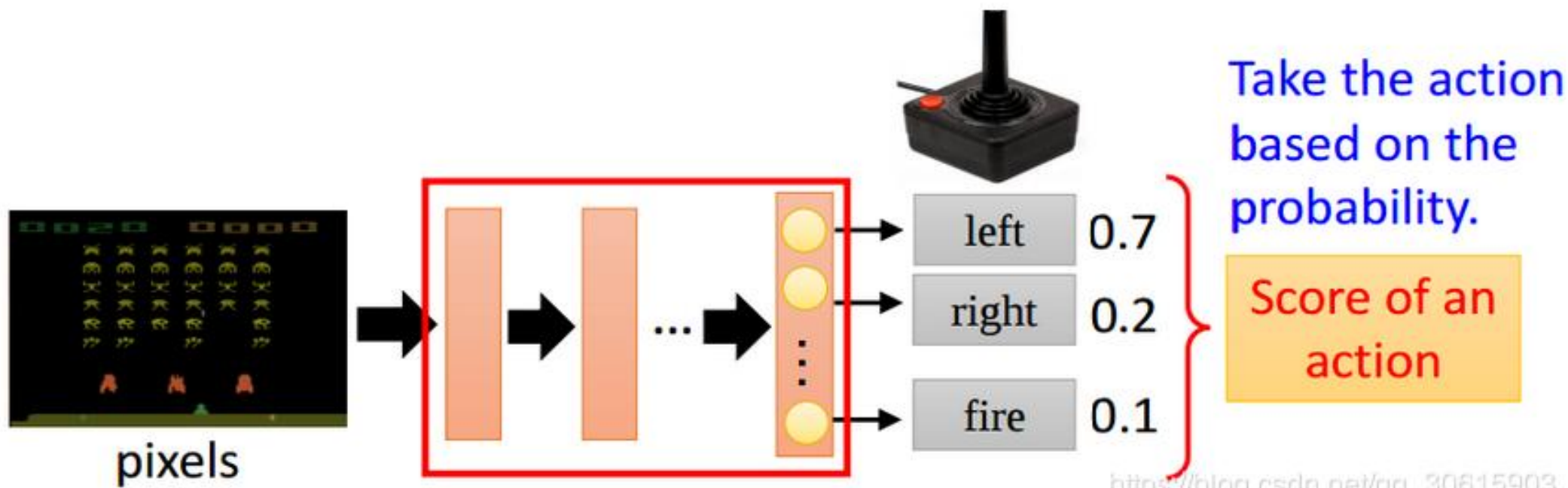
Mnih et al. proposed the DQN model, which combines convolutional neural networks with Q-learning algorithm, pioneering deep reinforcement learning. This model solves the discrete state limitation of Q-learning and utilizes neural networks to process continuous states.

● DQN architecture explanation

DQN uses CNN as a function approximator to input raw pixel data and output expected action returns (Q-values), thereby processing complex visual input tasks, such as Atari games, demonstrating powerful capabilities.

● DQN training algorithm

DQN learns value functions through convolutional neural networks, combined with Q-learning and gradient descent optimization, to achieve efficient training. Simultaneously introducing experience replay and target network mechanisms for breaking correlations and stabilizing training, respectively.



https://blog.csdn.net/qq_30615903

On-Policy RL vs Off-Policy RL

1. Q-Learning:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t))$$

2. SARSA:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

On Policy

Only interactive learning is performed with the same environment, and the sampling process is still generated by the same policy without using samples generated by other policies. SARSA and PPO belong to this category.

Off Policy

A method that can learn from one's own environment experience or gain experience from other environments, using data generated by other policies to update another policy. DQN and DDPG belong to this category.

Policy Gradient

function REINFORCE

Initialise θ arbitrarily

for each episode $\{s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\} \sim \pi_\theta$ **do**

for $t = 1$ to $T - 1$ **do**

$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s_t, a_t) v_t$

end for

end for

return θ

end function

Algorithm 1 “Vanilla” policy gradient algorithm

Initialize policy parameter θ , baseline b

for iteration=1, 2, ... **do**

 Collect a set of trajectories by executing the current policy

 At each timestep in each trajectory, compute

 the *return* $R_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$, and

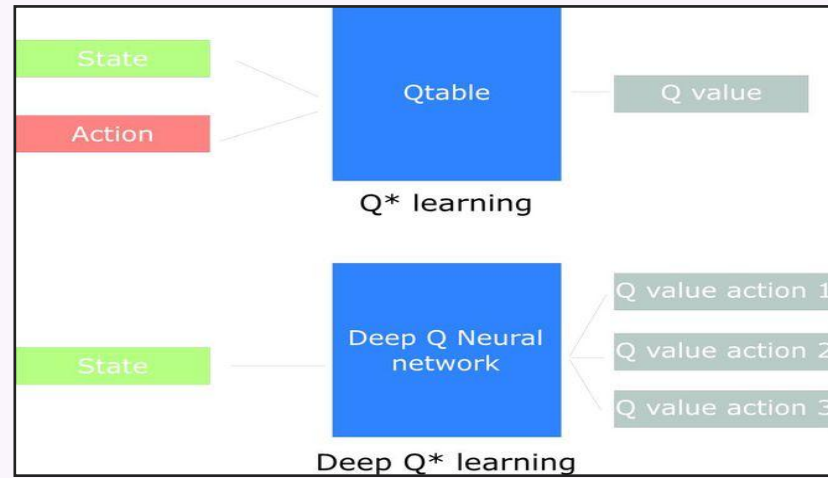
 the *advantage estimate* $\hat{A}_t = R_t - b(s_t)$.

 Re-fit the baseline, by minimizing $\|b(s_t) - R_t\|^2$,
 summed over all trajectories and timesteps.

 Update the policy, using a policy gradient estimate $\hat{g}$,
 which is a sum of terms $\nabla_\theta \log \pi(a_t | s_t, \theta) \hat{A}_t$

end for

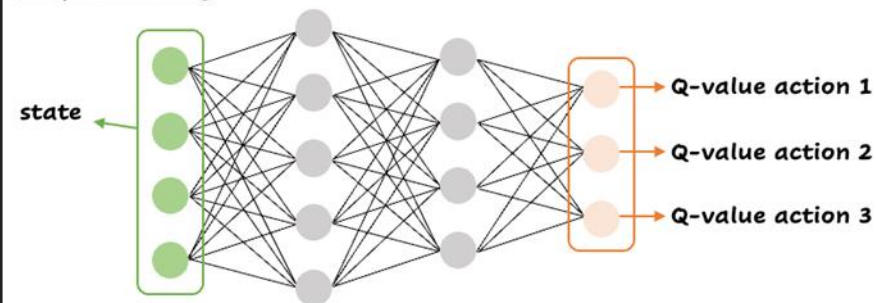
Q Learning vs. DQN



Tabular Q-learning

States	Actions			
	1	2	...	n
0	$Q(0,1)$	$Q(0,2)$	...	$Q(0,n)$
1	$Q(1,1)$	$Q(1,2)$	...	$Q(1,n)$
...	...	...	...	...
m	$Q(m,1)$	$Q(m,2)$	...	$Q(m,n)$

Deep Q-learning



Fixed Target Q Network

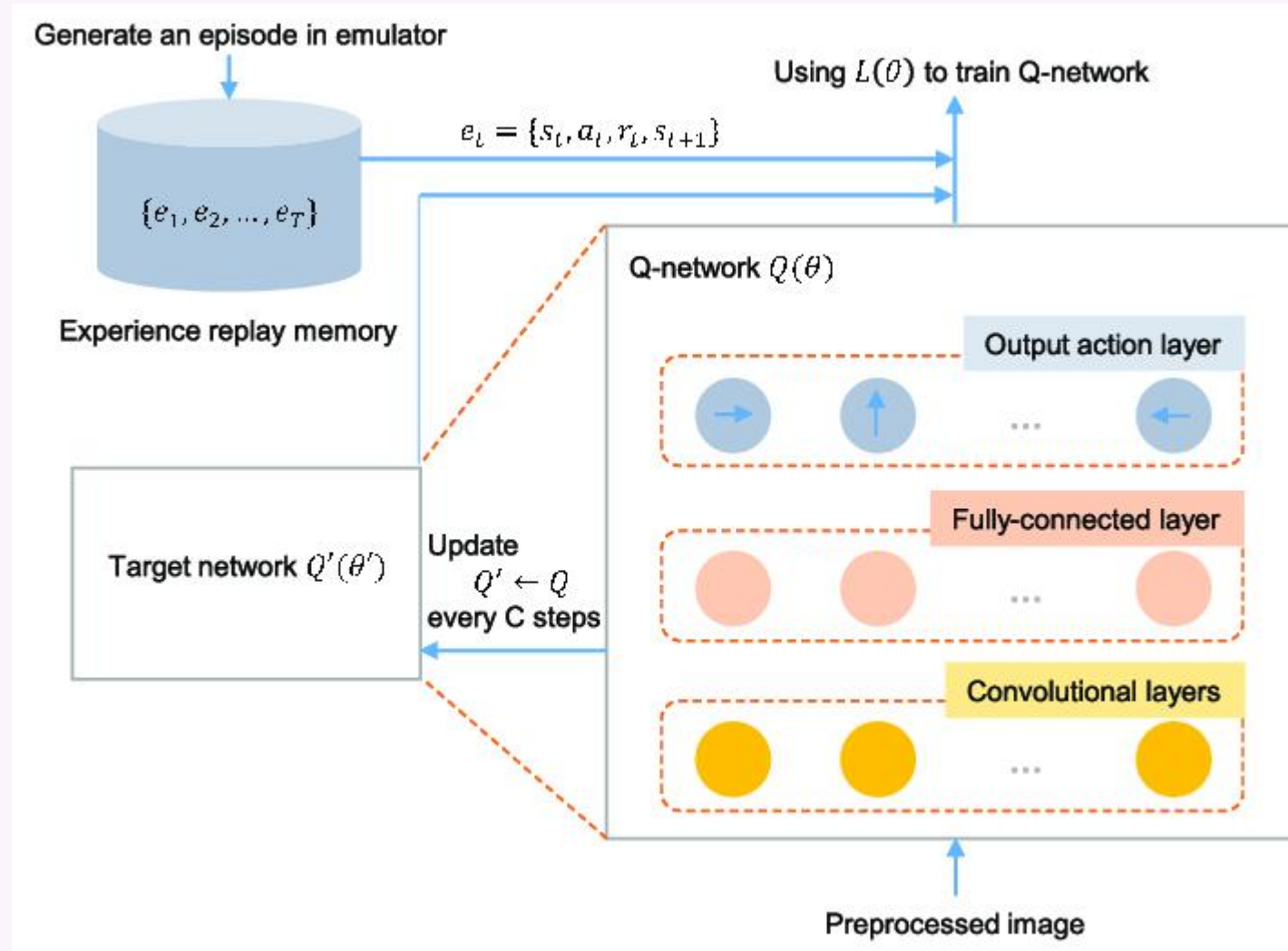
Implementation

Create a separate target Q network by replicating the main Q network, keeping the target network fixed for N iterations, and calculate the target Q value by passing the next state to the fixed target network.

Benefits

By keeping the target fixed, it improves training stability, avoids accumulated errors in recursive Bellman backups, allows the main Q-net to freely change without divergence, and provides consistent targets for training over iterations.

DQN Architecture



Analysis of DQN instability problem

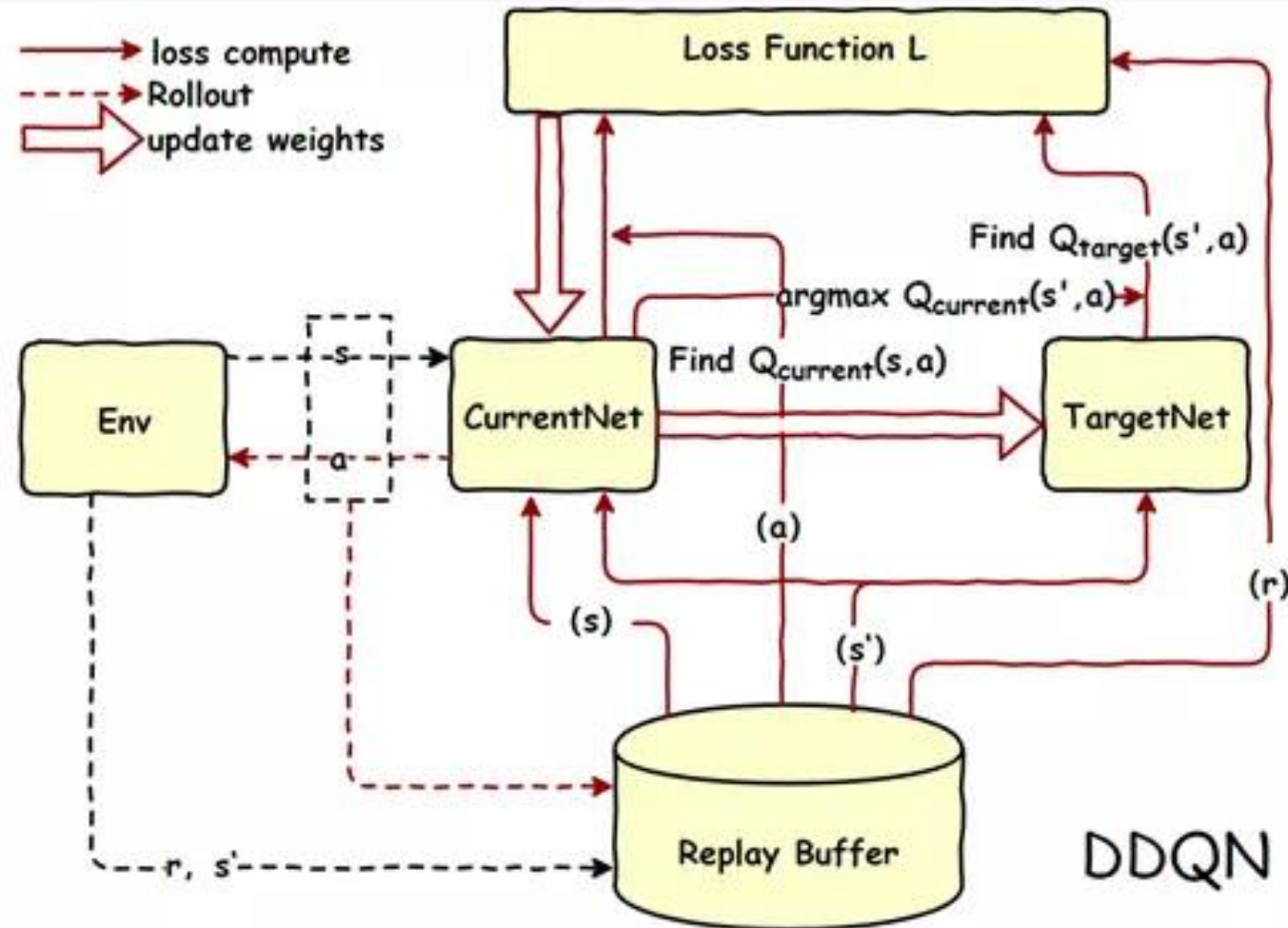
Overestimating Q-value

The Max operator leads to systematic overestimation, non-stationary objectives refer to using the same Q-network for action selection and evaluation, and correlation in data refers to using continuous samples with function approximation.

Deadly Triad

Function Approximation+Bootstrap+Off Policy Learning. Function approximation introduces bias, bootstrapping exacerbates overestimation, and off policy amplifies bias, forming a vicious cycle.

Double DQN



$$L = (r + \gamma Q_{target}(s', \text{argmax}_a Q_{current}(s', a)) - Q_{current}(s, a))^2$$

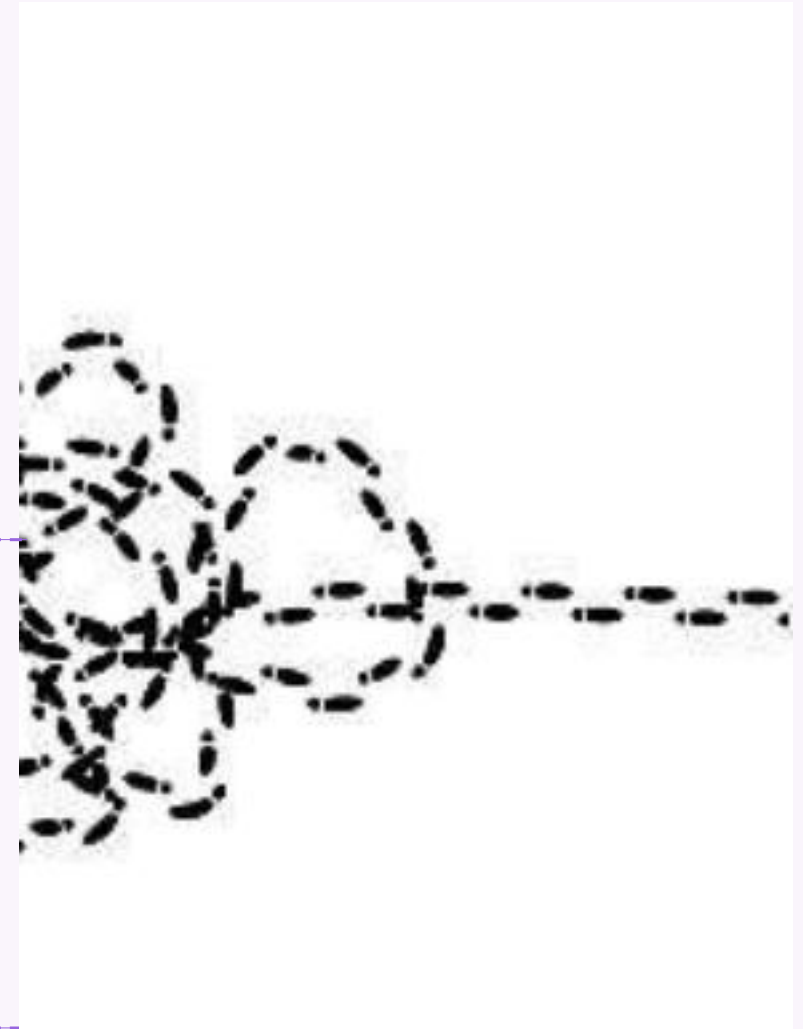
Double DQN Advantage

Target Q-value calculation

$y = r + \gamma \max_a Q(s', a; \theta^-)$ in DQN, $y = r + \gamma Q(s', \arg\max_a Q(s', a; \theta^-); \theta^-)$ in Double DQN;
Reduce overestimation bias.

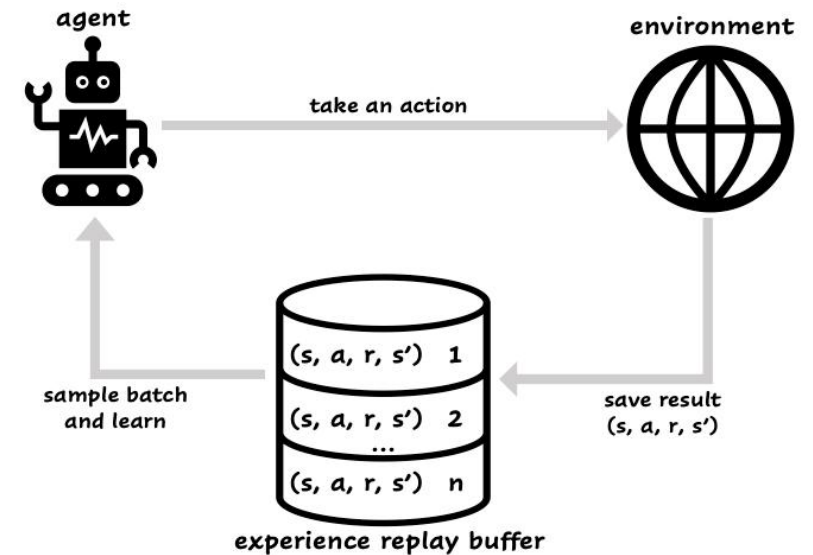
Advantages

The learning of Double DQN is more stable and reliable, improving performance and achieving higher scores in Atari games, which is a simple improvement to the DQN algorithm.



Experience Replay Mechanism

- **Experience replay:** In traditional reinforcement learning, agents continuously collect samples from the environment and immediately use these samples for learning, which may lead to high temporal correlation between samples and affect the stability of learning.
- **Experience replay mechanism:** By storing the experience of each step and randomly sampling a small batch of experience for training each time the strategy is updated, the temporal correlation between samples is broken, improving the efficiency and stability of model training.
- **Workflow:** Collect experience and store it in a replay buffer, randomly sample Mini Batch for training, and update DQN model weights. Repeat this process until the model optimization is completed.
- **Benefits:** Avoiding oscillations in related sequences, more efficient data utilization, mixing updates from different time steps, supporting batch training of neural networks, improving model training efficiency and stability.



DQN pseudocode implementation

Initialization



Initialize replay buffer D (storing experience) and Q -network (random weights); Set exploration rate ϵ , ϵ_decay , and ϵ_min (exploration rate and its attenuation and minimum threshold).

Training process



Randomly initialize the target network;
Execute actions, store experience, sample Mini Batch, and update Q network in the main loop; Update the target network after each round or specific number of steps;
Attenuation exploration rate from ϵ to ϵ_min .

DQN pseudocode implementation

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ε select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

End For

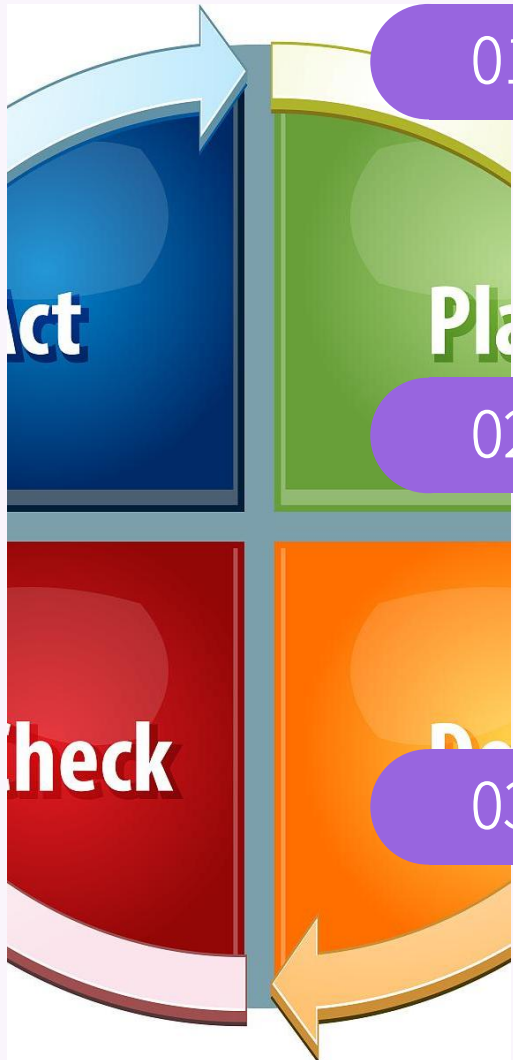
End For



03

Actor Critic Algorithm System

Overview of Actor Critic Principle



01

Actor Critic Reinforcement Learning

A reinforcement learning algorithm that combines policy networks (Actor) and value networks (Critic) to optimize policies through environmental interaction, action selection, reward acquisition, and value evaluation.

02

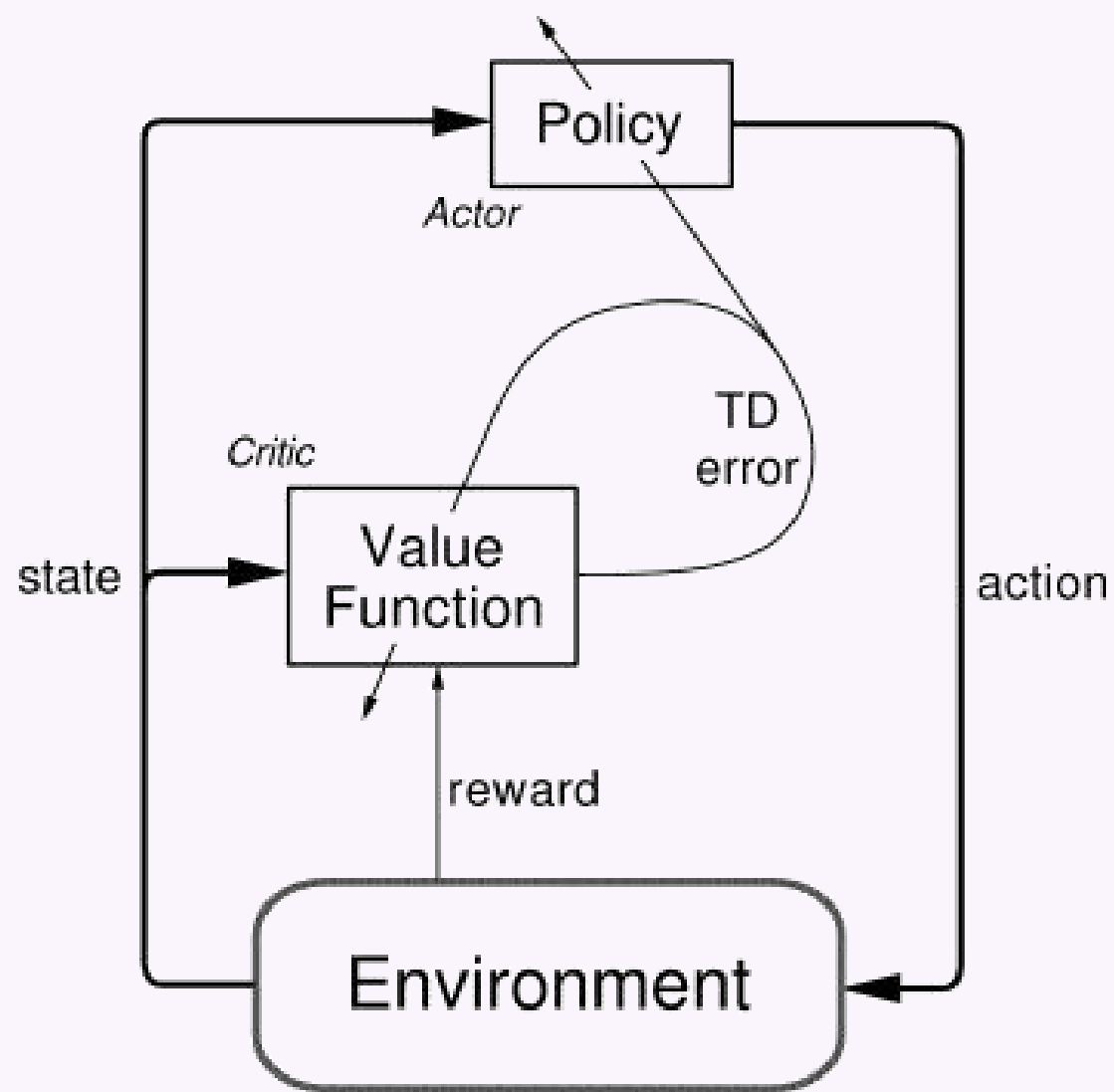
Strategic Networks and Value Networks

Strategic networks are responsible for selecting actions based on the current state, generating probability distributions or specific actions for the actions; Value network evaluates the quality of actions and calculates the value of states or state action pairs.

03

The working principle of Actor Critic

The agent interacts with the environment, the Actor selects actions, Critic evaluates their value, guides policy updates, maximizes cumulative rewards, and repeats interactions until the policy converges or achieves training objectives.



Classification and analysis of common algorithms

A2C algorithm

The random strategy is updated synchronously with the value function, introducing an advantage function to reduce variance and improve stability. It is suitable for continuous control, synchronous parallel acceleration training, and performs well in stability and convergence speed.

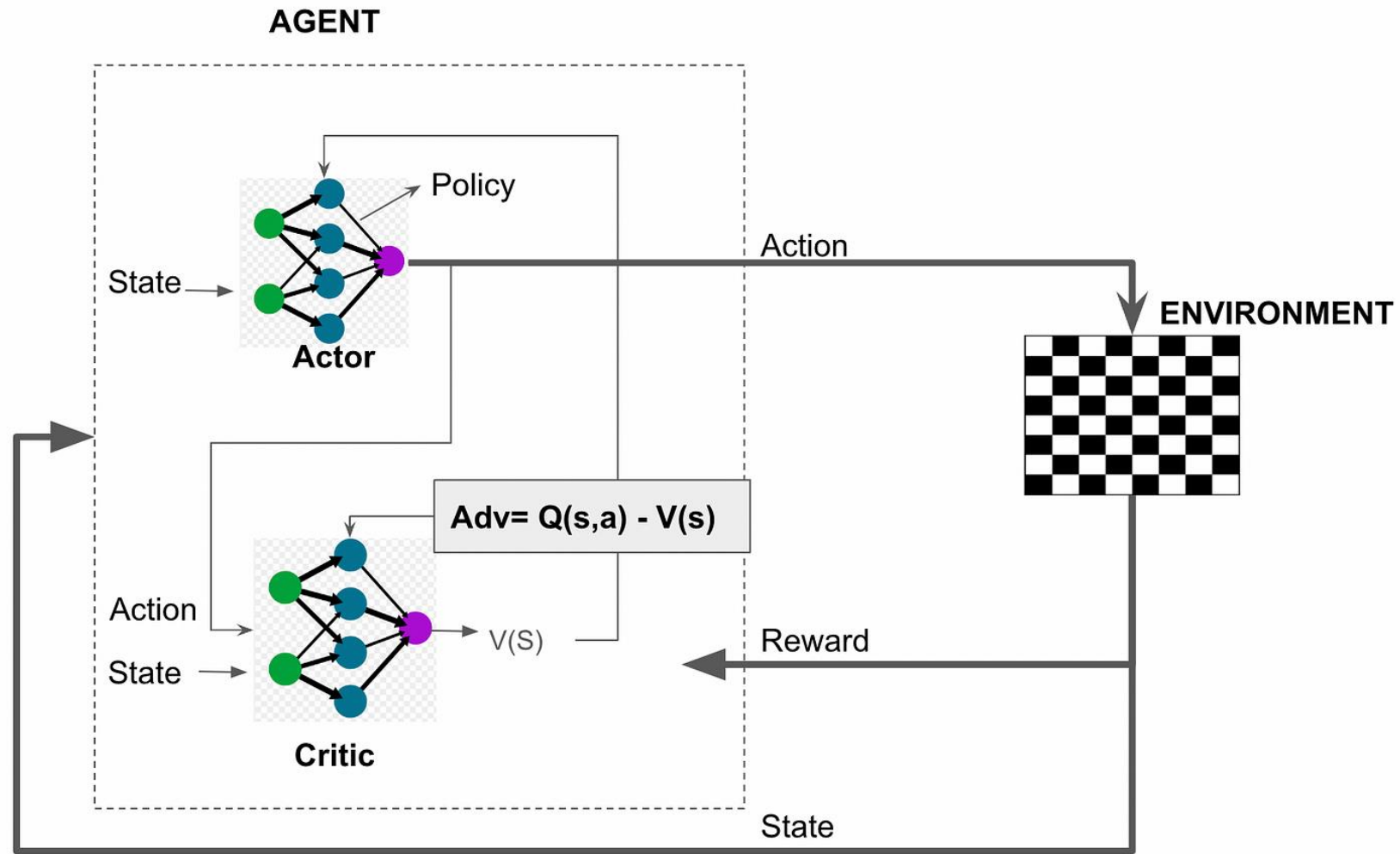
A3C algorithm

Asynchronous updates of random policies and value functions, supports parallel computing, improves training efficiency, suitable for multi environment parallel training scenarios, and achieves fast policy optimization through asynchronous updates.

DDPG algorithm

combining deterministic strategies with Q-functions, suitable for continuous action spaces, processing high-dimensional state spaces through deep neural networks, updating single step strategies, and efficiently handling dynamic and complex environments.

A2C Architecture



Classification and analysis of common algorithms

TD3 algorithm

Built on the basis of DDPG, optimized for the problem of overestimation, using a double-Q network structure to reduce bias and improve algorithm robustness, suitable for learning tasks in continuous action spaces.

PPO algorithm

a method based on policy gradient, which ensures stability by limiting the policy update step size, and combines reinforcement learning techniques with human feedback to optimize the training model, suitable for robust optimization in multiple environments.

Advantage Actor-Critic

Overview of A2C Algorithm

The Actor Critic (A2C) algorithm combines the Actor Critic framework with the advantage function to estimate the superiority of actions and reduce the variance of policy gradient estimation by introducing the advantage function.

Advantages of A2C algorithm

A2C algorithm has excellent stability and convergence speed, especially in continuous control. The synchronous parallel mechanism accelerates training, allowing multiple worker threads to run synchronously and efficiently collect data to update network parameters.

Actor Critic framework

The Actor network generates the probability distribution of actions based on the current state, while the Critic network evaluates the value of the state and predicts expected returns. The two work together to optimize strategy selection and value evaluation.

Advantage Actor-Critic

Advantage function

$A(s)$, where $Q(s, a)$ represents the Q value for executing action a in state s .

Loss function

consists of three parts: strategy loss, value loss, and entropy loss. These three loss functions jointly guide the training process of the network, updating network parameters through backpropagation algorithm to optimize strategy and value.

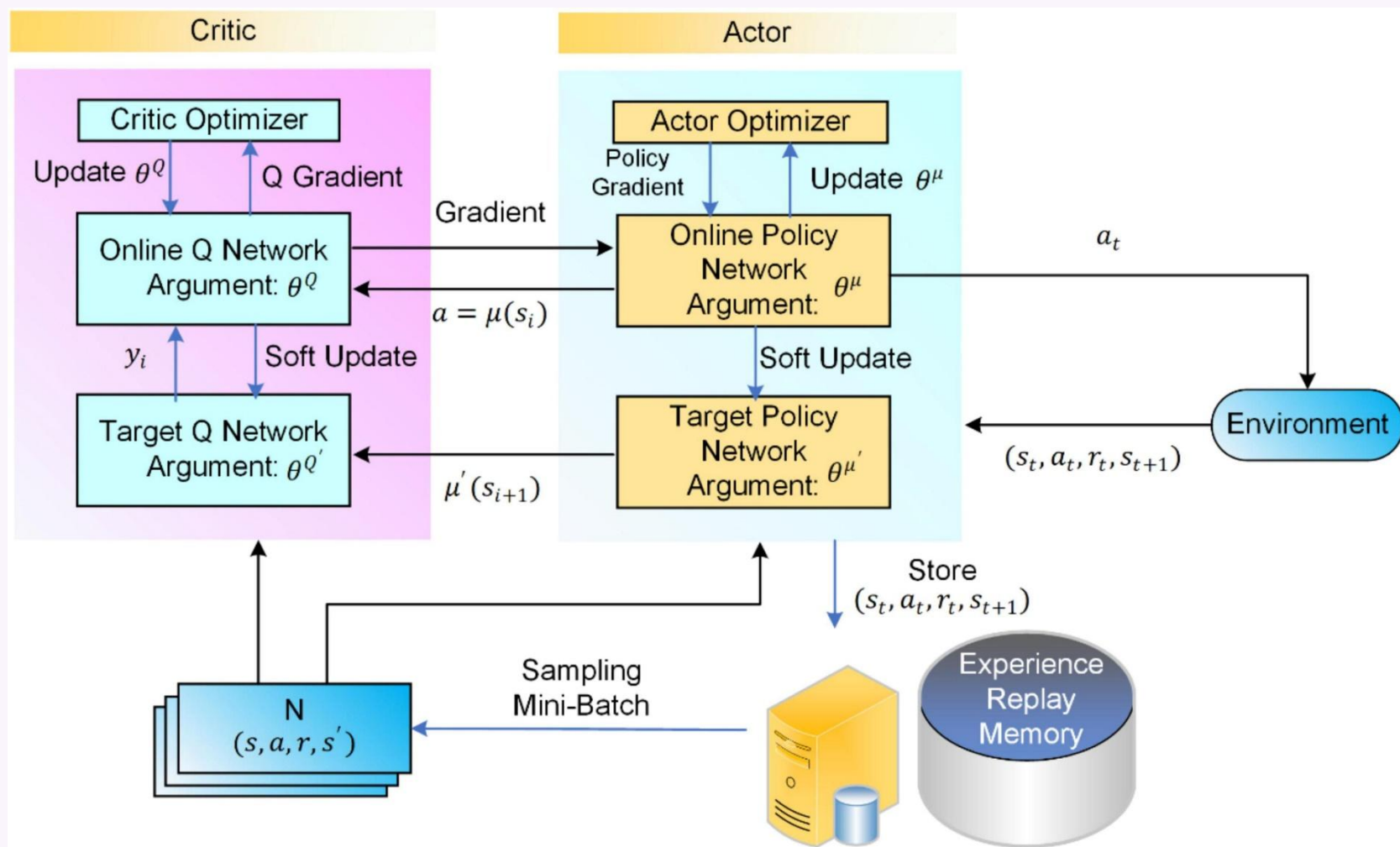
04

Deep analysis of DDPG algorithm

Comparative Study of DPG and SPG

- **DPG optimization for determinism:** Deterministic policy gradient (DPG) aims to optimize deterministic policies by directly outputting the optimal action, especially suitable for continuous action spaces, and can efficiently handle complex action selection problems.
- **SPG optimization randomness:** Stochastic Policy Gradient (SPG) focuses on optimizing random policies by indirectly selecting actions through the probability distribution of output actions, making it more suitable for discrete action spaces and providing diverse action choices.
- **Action selection differences:** DPG accurately selects specific actions, with clear and efficient strategies; SPG, on the other hand, selects actions based on probability to increase exploration, but may introduce additional uncertainty. Both have their own advantages and are suitable for different scenarios.
- **Application areas:** DPG is widely used in robotics technology and games that require fine motion control due to its efficient strategy for processing continuous action spaces; SPG excels in discrete decision-making scenarios such as chess, providing diverse action choices.

DDPG Architecture



Comparison between DQN and DDPG

DQN

Processing discrete low dimensional operation spaces. Find the maximum value of the behavior value function for each iteration. End to end learning; Limited to discrete operations only. DQN cannot output the behavior value of each operation. Exponential growth with motion freedom.

DDPG

Inheriting the deterministic policy gradient method, combining the advantages of DQN and DPG, effectively handling high-dimensional inputs and continuous action spaces, jointly optimizing policies and value functions, exploring and utilizing balance, and constructing an efficient reinforcement learning framework.

Ornstein Uhlenbeck process

OU process mean regression

The Ornstein Uhlenbeck process exhibits mean regression characteristics, with the system approaching the long-term mean over time. The volatility and mean regression speed are controlled by parameters, providing accurate models for fields such as finance and physics.

Widely applied and influential in various fields

OU processes are widely used in physics, finance, biology, and other fields. Their mean regression characteristics provide an important perspective for understanding the dynamics of complex systems, helping to predict and optimize decisions.

Ornstein-Uhlenbeck (OU)

The Ornstein-Uhlenbeck process is defined by the following stochastic differential equation (SDE):

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t$$

where:

- X_t is the state of the process at time t .
- θ is the rate of mean reversion.
- μ is the long-term mean level.
- σ is the volatility parameter.
- W_t is a standard Wiener process (or Brownian motion).

Implementation details of DDPG algorithm

- **Deep Deterministic Policy Gradient:** The DDPG algorithm combines deep learning with deterministic policy optimization, processes high-dimensional state space through deep neural networks, outputs deterministic actions, and is suitable for continuous action environments.
- **Advantages of Single Step Policy Update:** The DDPG algorithm updates policies after each environment interaction, achieving single step policy network updates and improving policy optimization efficiency, especially suitable for dynamic and complex environments.
- **Integrate efficient learning framework:** DDPG Actor Critic framework integrates strategy evaluation and improvement to achieve efficient and stable learning; The Pre AC framework provides intuitive strategy optimization methods, laying the foundation for algorithms such as DQN.
- **Improvement and extension of DDPG:** DDPG combines DQN with DPG to successfully process high-dimensional inputs, inherits deterministic policy gradient methods, and effectively extends to continuous action spaces, demonstrating strong generality.

DDPG Algorithm

Definitions and Notations

- **State (s):** The current position of the agent.
- **Action (a):** The movement of the agent (continuous value).
- **Reward (R):** The reward received after taking an action.
- **Policy ($\pi_\theta(s)$):** The actor network that outputs the action given the state, parameterized by θ .
- **Q-value ($Q(s, a)$):** The critic network that evaluates the action given the state, parameterized by ϕ .
- **Target Networks:** θ' and ϕ' are the parameters of the target actor and critic networks, respectively.
- **Replay Buffer:** A buffer that stores past experiences (s, a, R, s') .

1. **Initialize** the actor network $\pi_\theta(s)$ and critic network $Q_\phi(s, a)$ with random parameters θ and ϕ .
2. **Initialize** the target networks $\theta' \leftarrow \theta$ and $\phi' \leftarrow \phi$.
3. **Initialize** the replay buffer.
4. **For** each episode:
 1. **Initialize** the starting state s .
 2. **For** each step of the episode:
 1. Select action $a = \pi_\theta(s) + \mathcal{N}$, where $\mathcal{N}$ is noise for exploration.
 2. Execute action a , observe reward R and next state s' .
 3. Store the transition (s, a, R, s') in the replay buffer.
 4. Sample a random mini-batch of transitions (s_i, a_i, R_i, s'_i) from the replay buffer.
 5. **Critic Update:**
$$y_i = R_i + \gamma Q_{\phi'}(s'_i, \pi_{\theta'}(s'_i))$$
$$L = \frac{1}{N} \sum_i (y_i - Q_\phi(s_i, a_i))^2$$
Update ϕ by minimizing the loss L .
 6. **Actor Update:**
$$\nabla_\theta J \approx \frac{1}{N} \sum_i \nabla_a Q_\phi(s_i, a)|_{a=\pi_\theta(s_i)} \nabla_\theta \pi_\theta(s_i)$$
Update θ using the sampled policy gradient.
 7. **Target Networks Update:**
$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$
$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$
3. **Until** s is a terminal state.

DDPG Example

Example Computation

Let's consider a simple example with the following initial parameters

- Learning rate for actor (α_θ) = 0.001
- Learning rate for critic (α_ϕ) = 0.001
- Discount factor (γ) = 0.99
- Soft update parameter (τ) = 0.005

Initialize Networks and Replay Buffer

- Actor network $\pi_\theta(s)$ with random parameters θ .
- Critic network $Q_\phi(s, a)$ with random parameters ϕ .
- Target networks $\theta' \leftarrow \theta$ and $\phi' \leftarrow \phi$.
- Replay buffer initialized.

Episode 1

1. **Start State:** $s = 0$
2. **Select Action:** $a = \pi_\theta(0) + \mathcal{N}$
3. **Execute Action:** Move to state $s' = 0.1$, receive reward $R = 0.1$
4. **Store Transition:** $(0, a, 0.1, 0.1)$ in the replay buffer.

Sample Mini-Batch from Replay Buffer

Assume we sample the transition $(0, a, 0.1, 0.1)$.

Critic Update

1. Compute target value y :

$$y = R + \gamma Q_{\phi'}(s', \pi_{\theta'}(s')) = 0.1 + 0.99 \cdot Q_{\phi'}(0.1, \pi_{\theta'}(0.1))$$

Assume $Q_{\phi'}(0.1, \pi_{\theta'}(0.1)) = 0.2$, then:

$$y = 0.1 + 0.99 \cdot 0.2 = 0.298$$

2. Compute loss L :

$$L = (y - Q_\phi(0, a))^2$$

Assume $Q_\phi(0, a) = 0.1$, then:

$$L = (0.298 - 0.1)^2 = 0.039204$$

3. Update critic parameters ϕ to minimize loss L .

Actor Update

1. Compute policy gradient:

$$\nabla_\theta J \approx \nabla_a Q_\phi(s, a)|_{a=\pi_\theta(s)} \nabla_\theta \pi_\theta(s)$$

Assume $\nabla_a Q_\phi(0, a)|_{a=\pi_\theta(0)} = 0.5$ and $\nabla_\theta \pi_\theta(0) = 0.5$, then:

$$\nabla_\theta J = 0.5 \cdot 0.5 = 0.25$$

2. Update actor parameters θ using the policy gradient.

Target Networks Update

1. Update target actor network θ' :

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$$

2. Update target critic network ϕ' :

$$\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$$

DDPG pseudo code detailed explanation

01

Overview of pseudo code

Initialize network, target network, replay buffer; Loop selection action with noise, experience storage, and update status; Update the network after experience replay; If the termination condition is met, it will end. The DDPG core loop is clear.

02

DDPG algorithm core

The DDPG algorithm optimizes continuous action space learning through deep networks and empirical replay. The core loop consists of action selection with exploration noise, experience storage, and network updates, effectively addressing complex environmental challenges.

Application Cases of Automotive Driving

01

DDPG driving application

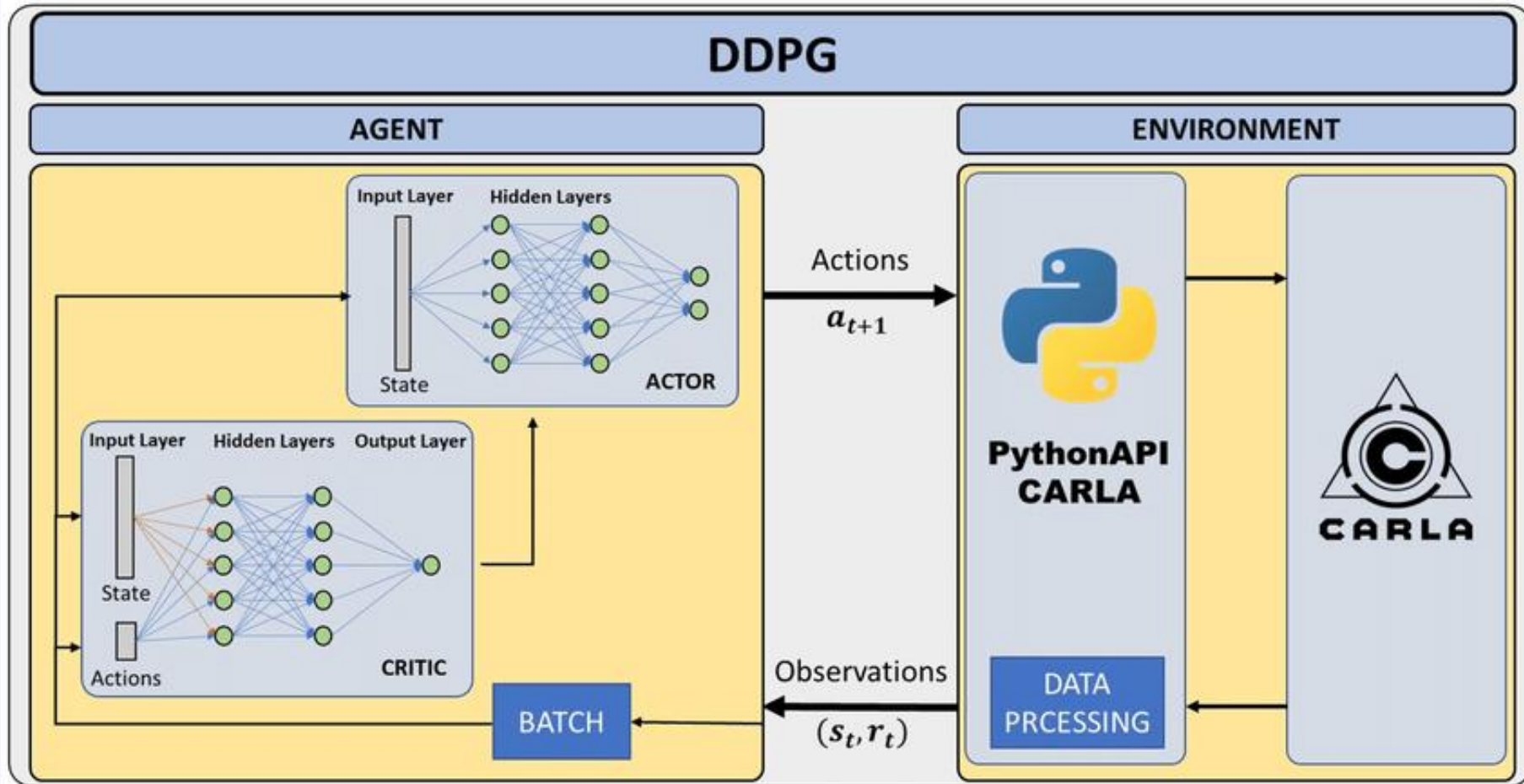
The DDPG algorithm demonstrates its potential in car driving, improving vehicle handling ability through simulation training. In complex road environments, DDPG can plan and adjust continuous actions such as vehicle speed and direction.

02

Adaptive cruise control

Combined with DDPG's adaptive cruise control system, the vehicle can intelligently respond to sudden road conditions, such as emergency braking or obstacle avoidance turning. The reinforcement learning mechanism makes vehicles more responsive and improves driving safety.

Auto Driving Car with DDPG



05

Practical Application of PPO Algorithm

The Core Idea of PPO algorithm



The core idea of PPO

The PPO algorithm aims to ensure the stability of the learning process by limiting the magnitude of policy updates, while utilizing the dominance function to guide policy optimization, balancing exploration and utilization, and improving sample efficiency.



Trimming mechanism

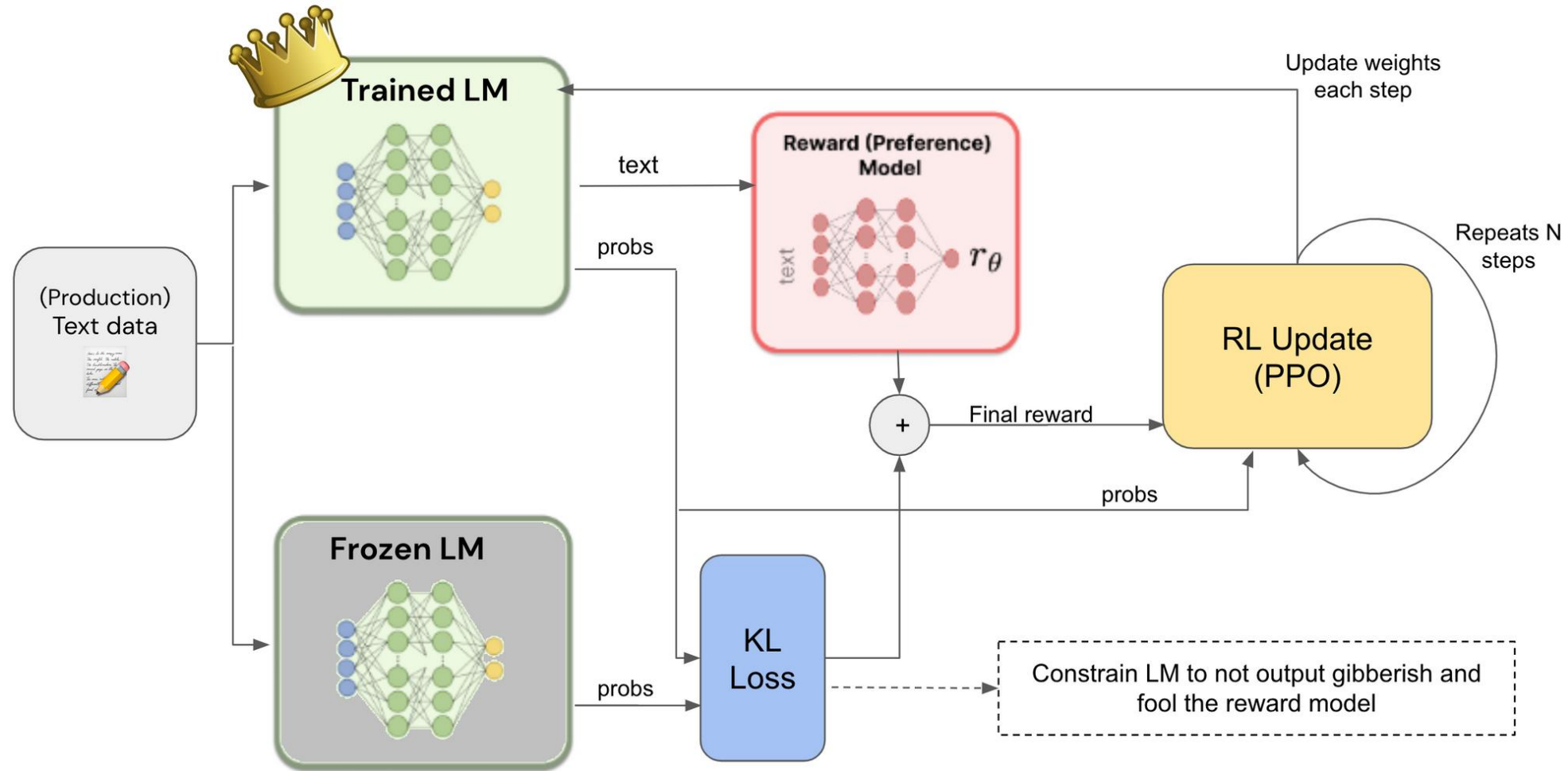
PPO restricts the policy update step size through pruning mechanism to prevent significant changes, ensure stability during training, and effectively prevent oscillation problems in policy gradient optimization.



Application areas

Due to its simplicity and efficiency, PPO algorithm is widely used in fields such as game AI, robotics technology, and large model optimization, achieving efficient solutions to complex tasks through reinforcement learning.

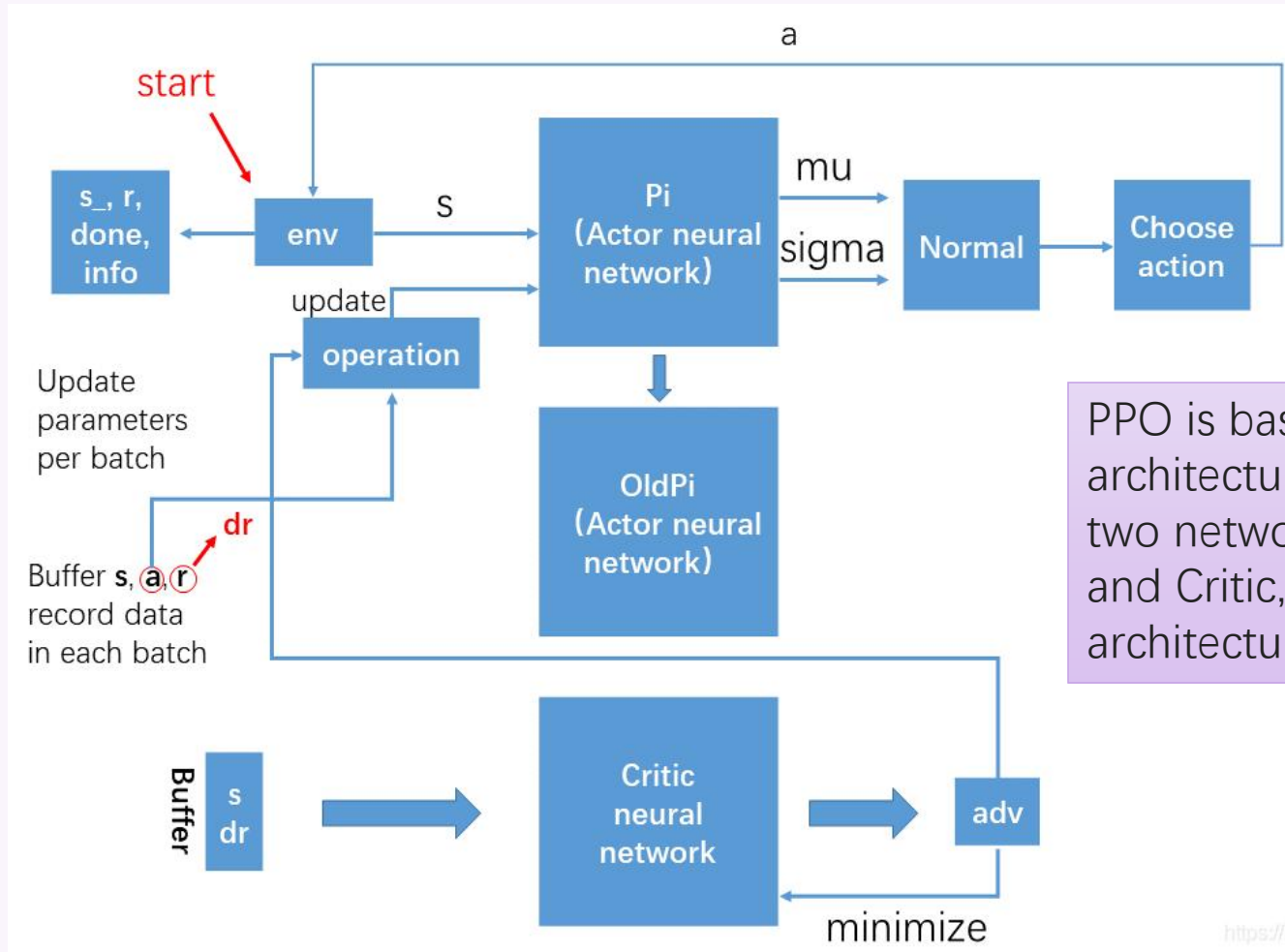
RLHF with PPO



Analysis of PPO Implementation Steps

- **Initialization and computational advantages:** In the PPO algorithm, the strategy network and value function are first initialized with random weights, and then the advantages are calculated to clarify the degree to which the return exceeds the prediction, providing guidance for strategy optimization.
- **Strategy update and loop:** Collect data to generate trajectories, use pruning techniques to update strategies, and optimize strategies through multiple random gradient rising epochs. This process is repeated, constantly iterating and improving strategies.
- **Crop mechanism:** The PPO algorithm introduces a crop mechanism by setting a crop threshold to limit the magnitude of policy updates, effectively preventing training instability caused by excessive policy changes and improving the stability and robustness of the algorithm.
- **Hyperparameter control:** The performance of PPO algorithm is affected by multiple hyperparameters, including learning rate, which controls the step size of policy and value function updates; The cutting threshold determines the trust zone range for policy optimization.

PPO Architecture



PPO is based on the AC architecture, and there are also two networks, which are Actor and Critic, because the AC architecture has a benefit.

PPO architecture and pseudocode

- **PPO is based on the AC architecture:** The PPO algorithm is based on the Actor Critic (AC) architecture and consists of two key networks, Actor and Critic. The choice of this architecture brings obvious advantages, such as parallel processing of strategy optimization and value evaluation.
- **Actor and Critic functions:** In the PPO algorithm, the Actor network is responsible for selecting actions based on the current state, that is, generating policies; The Critic network evaluates the quality of the actions chosen by the Actor, providing feedback by calculating the value of the state or state action pairs.
- **Overview of pseudo code:** PPO pseudo code demonstrates the core loop of the algorithm, including initialization, action selection with exploration noise, experience storage, and network updates. The algorithm stabilizes training in continuous action space through experience replay and target network.

PPO Algorithm

Definitions and Notations

- **State (s):** The current position of the agent.
- **Action (a):** The movement of the agent (continuous value).
- **Reward (R):** The reward received after taking an action.
- **Policy ($\pi_\theta(a|s)$):** The policy network that outputs the probability of taking action a given state s , parameterized by θ .
- **Value Function ($V_\phi(s)$):** The value network that estimates the value of state s , parameterized by ϕ .
- **Old Policy ($\pi_{\theta_{\text{old}}}(a|s)$):** The policy used to generate the data, parameterized by θ_{old} .
- **Clip Range (ϵ):** The range within which the policy ratio is clipped.

Steps of the PPO Algorithm

1. **Initialize** the policy network $\pi_\theta(a|s)$ and value network $V_\phi(s)$ with random parameters θ and ϕ .
2. **For** each iteration:
 1. **Collect Trajectories:** Run the policy $\pi_{\theta_{\text{old}}}$ in the environment for T timesteps and collect trajectories (s_t, a_t, R_t, s_{t+1}) .
 2. **Compute Advantages:** Compute the advantage estimates $\hat{A}_t$ using the collected trajectories.
 3. **Compute Targets:** Compute the target value estimates R_t .
 4. **Optimize Policy:**
 1. **For** each epoch:
 1. **For** each mini-batch:
 1. Compute the ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$.
 2. Compute the clipped objective:
$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$
 3. Update the policy parameters θ by maximizing $L^{\text{CLIP}}(\theta)$.
 5. **Optimize Value Function:**
 1. **For** each epoch:
 1. **For** each mini-batch:
 1. Compute the value loss:
$$L^{\text{VF}}(\phi) = \frac{1}{N} \sum_t (V_\phi(s_t) - R_t)^2$$
 2. Update the value network parameters ϕ by minimizing $L^{\text{VF}}(\phi)$.
 6. **Update Old Policy:** $\theta_{\text{old}} \leftarrow \theta$.

PPO Example

Let's consider a simple example with the following initial parameters:

- Learning rate for policy (α_θ) = 0.001
- Learning rate for value function (α_ϕ) = 0.001
- Discount factor (γ) = 0.99
- Clip range (ϵ) = 0.2

Initialize Networks

- Policy network $\pi_\theta(a|s)$ with random parameters θ .
- Value network $V_\phi(s)$ with random parameters ϕ .
- Old policy network $\pi_{\theta_{\text{old}}}(a|s)$ initialized to $\pi_\theta(a|s)$.

Collect Trajectories

Assume we collect the following trajectory:

- State $s_0 = 0$
- Action $a_0 = 0.1$
- Reward $R_0 = 0.1$
- Next state $s_1 = 0.1$

Compute Advantages

Assume the advantage estimate $\hat{A}_0 = 0.2$.

Compute Targets

Assume the target value estimate $R_0 = 0.1$.

Optimize Policy

1. Compute Ratio:

$$r_0(\theta) = \frac{\pi_\theta(0.1|0)}{\pi_{\theta_{\text{old}}}(0.1|0)} = 1.1$$

2. Compute Clipped Objective:

$$L^{\text{CLIP}}(\theta) = \min(1.1 \cdot 0.2, \text{clip}(1.1, 0.8, 1.2) \cdot 0.2) = \min(0.22, 0.22)$$

3. Update Policy Parameters:

Update θ by maximizing $L^{\text{CLIP}}(\theta)$.

Optimize Value Function

1. Compute Value Loss:

$$L^{\text{VF}}(\phi) = (V_\phi(0) - 0.1)^2$$

Assume $V_\phi(0) = 0.05$, then:

$$L^{\text{VF}}(\phi) = (0.05 - 0.1)^2 = 0.0025$$

2. Update Value Network Parameters:

Update ϕ by minimizing $L^{\text{VF}}(\phi)$.

Update Old Policy

$$\theta_{\text{old}} \leftarrow \theta.$$



06

Model based reinforcement learning

Mode-based vs Model-Free

01

Mode free RL

does not rely on environmental models, learns directly from experience and action, has the advantage of being easy to implement, focuses on observable results, is suitable for dynamic and unknown environmental scenarios, and can flexibly adapt to environmental changes.

02

Mode based RL

Utilizing internal models of the environment for learning has the advantage of improving efficiency, reducing the need for extensive exploration, and is suitable for situations where the population has clear and pre-existing knowledge of environmental dynamics.

Dyna-Q Algorithm

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

DQN vs Dyna-Q

DQN

Combining convolutional neural networks with Q-learning, pioneering deep reinforcement learning, processing high-dimensional data through neural networks, solving the discrete state limitation of traditional Q-learning, and providing a solution for complex visual input tasks.

Dyna-Q

Combining model learning and Q-learning, utilizing simulation experience to accelerate learning, and combining planning to improve efficiency. It has higher efficiency than standard DQN samples and may converge faster, making it suitable for planning efficiency improvement tasks.

07

AlphaGo application case

AlphaGo Core Architecture



The role of reinforcement learning

The core of AlphaGo is reinforcement learning, which continuously refines strategies through self play, achieving a high level of improvement in Go skills, challenging and surpassing top human Go players.

Supervised learning stage

AlphaGo conducted supervised learning through a large amount of human expert game data, thus initially mastering the basic rules and strategies of Go, laying the foundation for subsequent training.

Self play stage

AlphaGo's core strategy is self play, constantly refining strategies through confrontation with oneself, and achieving self growth, which is a key step in becoming a top player.

AlphaGo Core Architecture

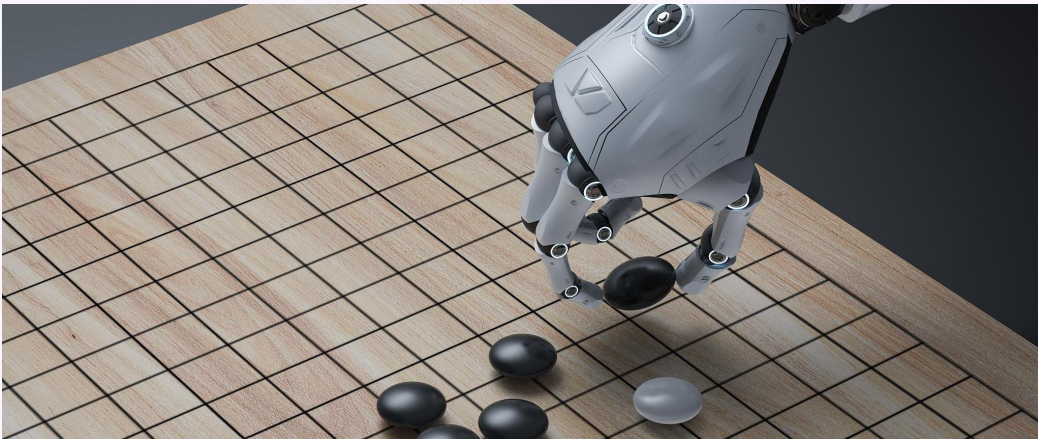


Strategy improvement

During self play, AlphaGo can continuously improve the accuracy of its value prediction, enabling it to make more precise and powerful decisions when playing chess.

Value refinement

Through self play, AlphaGo can also improve the accuracy of winning probability assessment, thereby better guiding actual games and increasing the probability of winning.



Basic principles of MCTS

01

Selection stage

Starting from the root node, select the most promising child nodes for exploration based on the UCB (Upper Confidence Bound) formula until reaching unexplored or terminal nodes.

02

Expansion phase

If the leaf node is not the terminating node, create a child node and select one to expand, in order to enrich the search tree structure and provide more path options for random simulation.

03

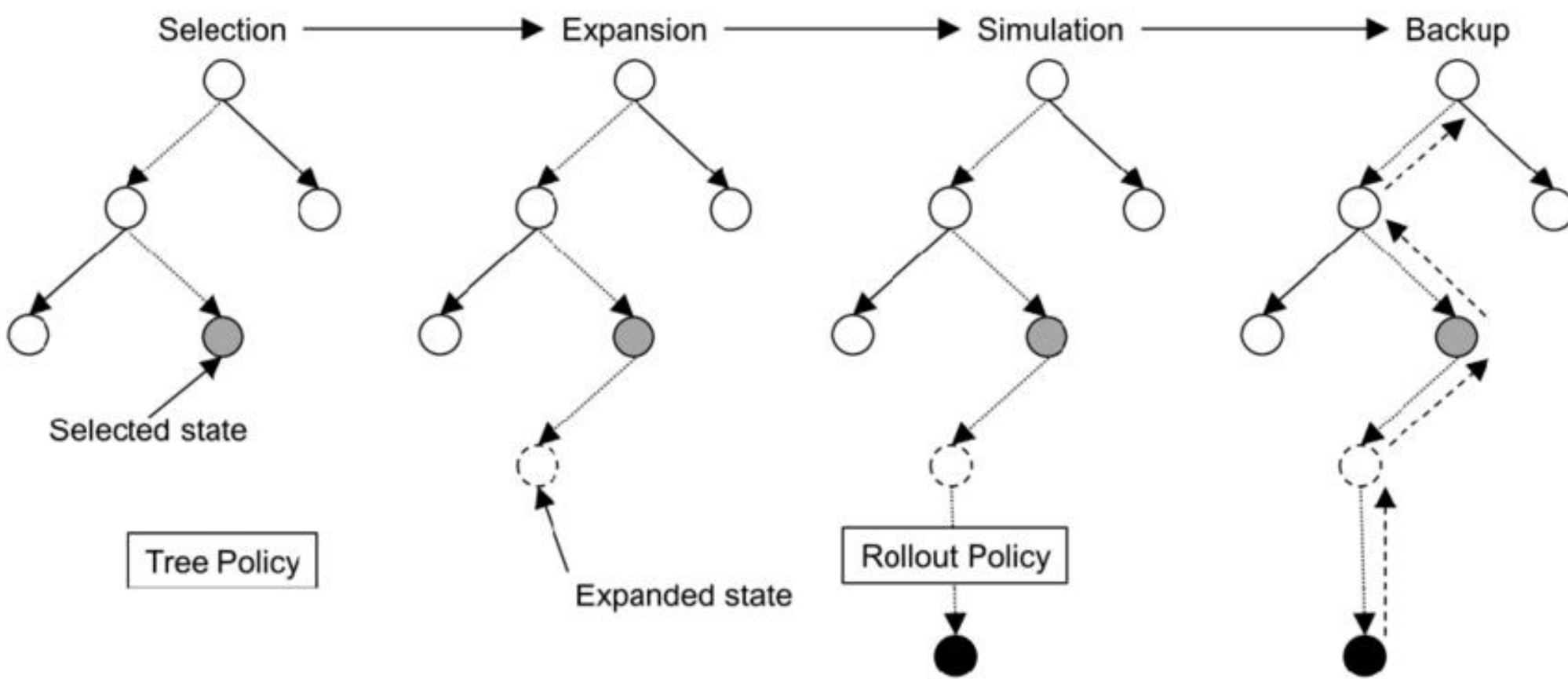
Simulation phase

Starting from the expanded nodes, perform a random simulation game until it ends or meets the termination conditions. The simulation process follows established rules, such as Go rules, to evaluate the effectiveness of decisions.

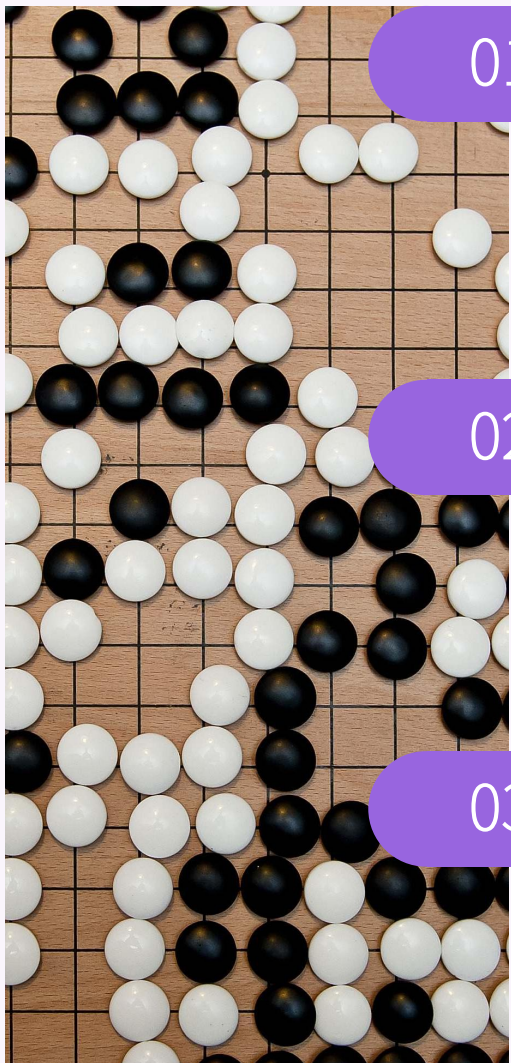
04

Feedback stage

Backpropagate the simulation results to the search tree, update the statistical information of each node, such as visit frequency, score, etc., to guide future search directions and optimize decision quality.



AlphaZero Algorithm



01

Initialization

When initializing the AlphaGoZero class, a neural network is constructed and MCTS (Monte Carlo Tree Search) is initialized, while setting parameters such as self play memory and training iteration times.

02

Self play

In each self play game, starting from the initial chessboard state, the MCTS search is executed in a loop to select the position of the player, update the chessboard state until the end of the game, and finally return the game data and results.

03

Training process

Through iterative self play, collect game data for neural network training. After each round of training, optimize the neural network using self playing data to improve chess strategy and evaluation ability.

AlphaGo Zero Architecture

